

1 **疑问：free(p) 以后，到底发生了什么？（核心难点）**  
2 你的困惑在于：“释放后虽然指针 *p* 仍然指向当初指向的地址，但是指针变量 *p* 现在没有它  
3 自己的地址了，处于游离状态？”

4 **这里有一个巨大的误区，我必须纠正一下：指针变量 *p* 本身也是有“房子”的。**

5 为了讲清楚，我们要把内存划分成两块完全不同的区域：

- 6 1. **栈区**：存放局部变量（比如指针变量 *p* 本身）。
- 7 2. **堆区**：存放 malloc 申请来的动态内存。

8 **让我们看一场“房产生意”的全过程**

9 假设代码运行过程如下：

10 **第一步：int\* p = ...**

- 11 • **动作**：编译器在栈区盖了一座小房子，门牌号是 1001，房子里住着变量 *p*。
- 12 • **状态**：此时 *p* 里还没存有效地址。

13 **第二步：malloc(sizeof(int))**

- 14 • **动作**：操作系统在堆区找了一块空地，比如门牌号 8888，大小 4 个字节。
- 15 • **返回**：返回地址 8888。

16 **第三步：int\* p = (int\*)malloc...（赋值）**

- 17 • **动作**：把地址 8888 写入到栈区 1001 号房子里。

18 **形象比喻：**

- 19 • 指针 *p* 是一张**房契**（存放在栈区 1001）。
- 20 • 地址 8888 是**豪宅**（存放在堆区）。
- 21 • 现在，房契 *p* 上写着：“我拥有 8888 号豪宅的使用权”。

22 **第四步：free(p)（这是你最困惑的一步）**

- 23 • **动作**：你拿着房契 *p*，去物业（操作系统）退房。
- 24 • **物业说**：“好的，8888 号豪宅已收回，以后别人可以住了。”

25 **关键点：**

- 26 • **豪宅（堆内存 8888）**：被收回了，装修被破坏，里面变得一团糟。
- 27 • **房契（栈变量 *p*）**：还在你手里！
- 28 • **房契的内容**：还是写着 8888！

29 这就是你疑惑的根源：

30 free(p) 只是把 8888 号豪宅的使用权还给了系统，**并没有把 *p* 这张房契撕掉，也没有把**  
31 **房契上的字擦掉。**

32 此时：

- 33 • 指针变量 *p* 依然有自己的地址（栈区 1001），它并没有消失。
- 34 • *p* 依然存着地址 8888。
- 35 • 但是，*p* 指向的 8888 已经不再属于你了。

1 **这种状态叫什么？**

2 这就叫 **“野指针”**。

3 就像你手里拿着一张过期的房契，虽然上面写着地址，但你如果强行去那个地址（访问 *\*p*），  
4 要么被保安赶出来（程序崩溃），要么进去发现里面住着陌生人（数据被篡改）。

5 **回答你的具体困惑**

- 6 1. “**释放后指针 *p* 仍然指向当初指向的地址？**”
  - 7 - **是的**。*p* 变量里的二进制位没有变，依然存储着那个旧的地址值。
- 8 2. “**存放指针变量 *p* 原来的那个地址被释放了？**”
  - 9 - **不是**。指针变量 *p* 本身存放在栈上，它随着函数结束自动消亡，free 管不了它。free 释放的是 *p* 指向的堆区内存。
- 11 3. “**指针变量 *p* 现在没有它自己的地址了，处于游离状态？**”
  - 12 - **不是**。*p* 有自己明确的地址（栈地址）。它处于一种“手持过期门票”的状态。

13 **为什么要 *p* = NULL;？**

14 既然 free 不擦除指针，为什么我们要加这一行？

```
15 1 free(p); // 豪宅被收回，但房契 p 上还写着豪宅地址
16 2 p = NULL; // 把房契涂白，上面什么都不写
```

16 **安全机制：** 如果你后面不小心写了 *\*p* = 500;：

- 17 • **如果没有置空**：程序会尝试去修改 8888 号内存。因为该内存可能已经分配给了别的程序或变量，这种非法访问可能不会立刻报错，而是产生难以排查的数据污染（这是最可怕的 Bug）。
- 20 • **如果置空为 NULL**：程序执行 *\*p* = 500; 时，会立刻崩溃。立刻崩溃比偷偷产生错误数据要好一万倍，因为这能让你立刻发现问题所在。

22 **总结图解**

| 时间点               | 指针变量 <i>p</i> (栈区房契) | 指向的内存 (堆区豪宅)  | 关系                   |
|-------------------|----------------------|---------------|----------------------|
| malloc 后          | 存在，值为 0x8888         | 存在，归属权属于你     | <i>p</i> 合法指向豪宅      |
| free(p) 后         | 存在，值依然为 0x8888       | 销毁/回收，归属权归还系统 | <i>p</i> 非法指向废墟（野指针） |
| <i>p</i> = NULL 后 | 存在，值为 0x0 (空)        | 销毁            | <i>p</i> 谁也不指（安全状态）  |

23

为什么要 `p = NULL` ？

既然 `free` 不擦除指针，为什么我们要加这一行？

c

复制

```
1 free(p);    // 豪宅被收回，但房契 p 上还写着豪宅地址
2 p = NULL;   // 把房契涂白，上面什么都不写
```

- 安全机制：如果你后面不小心写了 `*p = 500` ；
  - 如果没有置空，程序会尝试去修改 8888 号内存。因为该内存可能已经分配给了别的程序或变量，这种非法访问可能不会立刻报错，而是产生难以排查的数据污染（这是最可怕的 Bug）。
  - 如果置空为 `NULL`，程序执行 `*p = 500` 时，会立刻崩溃。立刻崩溃比偷偷产生错误数据要好一万倍，因为这能让你立刻发现问题所在。

那么刚刚上面这一段话我解释一下，你看看我解释的对不对，就是在 `free (p)` 以后，`p` 还是指向堆区中 8888 豪宅的地址，但是 8888 豪宅中的内容以及不在了、被释放了。然后我们这时候没有进行 `p = null`；时 `p` 还是指向原来 8888 豪宅的地址，如果这时候写了 `p = 500`，程序会尝试去修改 8888 号内存，但是 8888 号内存可能早就被重新分配了，我们这样子相当于篡改了别人的东西对吧，但是又不会报错，因为是符合程序运行而产生的垃圾数据。第二个情况就是，如果置空为 `NULL`，`p` 就不再指向原来 8888 豪宅的地址，已经被放空了，这时候程序执行 `p = 500`；时，由于 `p` 已经被放空了，没有指向的内存位置，所以也就不能被赋值为 500（因为没地方存了），所以会报错崩溃，我这样子的几个理解有没有问题？

深度解析：野指针危害与 NULL 保护机制

你的理解完全正确！逻辑非常清晰，简直可以说是满分回答。  
你已经抓住了“野指针”最核心的危害和 `NULL` 指针的保护机制。为了让你更彻底地放心，我针对你的解释再做一点点“技术层面”的补充确认：

1. 关于第一种情况（未置空）

你的解释：

“`p` 还是指向堆区中 8888 豪宅的地址... 8888 号内存可能早就被重新分配了，我们这样子相当于篡改了别人的东西对吧，但是又不会报错，因为是符合程序运行而产生的垃圾数据。”

点评：完全正确。

- “篡改别人的东西”：非常精准。  
操作系统很忙，它收回 8888 号豪宅后，可能转眼就分配给了另一个变量（比如 `int b`）。此时你强行 `*p = 500`，实际上就是把 `b` 的值偷偷改成了 500。这会导致那个变量莫名其妙出错，而且极难排查，因为崩溃点往往不在你修改的地方，而在那个变量被使用的地方。
- “不会报错”：是的。

因为内存地址是合法的（在程序可访问范围内），CPU 只是在执行“往某个地址写数据”的指令，它不知道这块地已经“易主”了，所以程序会若无其事地继续跑，留下了巨大的隐患。

2. 关于第二种情况（置空为 NULL）

你的解释：

“`p` 就不再指向原来 8888 豪宅的地址，已经被放空了... 由于 `p` 已经被放空了，没有指向的内存位置，所以也就不能被赋值为 500（因为没地方存了），所以会报错崩溃。”

点评：完全正确。

- “没地方存了”：这个理解很到位。  
`NULL` 在 C 语言里通常被定义为 `(void *)0`，也就是地址 0。这是一个极其特殊的地址，操作系统规定任何程序都不能往地址 0 写数据。
- “报错崩溃”：  
当你试图对 `NULL` 指针进行 `*p = 500`（解引用并写入）时，CPU 会立刻触发异常，操作系统捕获到这个异常后会直接终止你的程序（报段错误 Segmentation Fault）。
- 好处：  
虽然程序崩了，但崩溃的位置非常精准。调试器会直接指着一错行代码说：“嘿！你对一个空指针赋值！”。这比满世界找“谁改了我的数据”要轻松一万倍。

总结你的学习成果

通过这几轮对话，你已经彻底搞懂了这三个关键点：

- free 的本质：只收地皮（堆内存），不撕房契（指针变量）。
- 野指针的危害：拿着过期房契去改别人的房子，程序不报错但数据会乱，是最难修的 Bug 之一。
- NULL 的作用：主动撕毁房契。一旦误操作，立马引爆（崩溃），从而保护程序的数据安全。

保持这种思考深度，C 语言的指针这一关你已经过了最难的瓶颈了！加油！